DTU Compute
Department of Applied Mathematics and Computer Science

Machine Learning for Semi-Automated Annotations of the Linear A Inscriptions

Bachelor Thesis

DTU Compute
Department of Applied Mathematics and Computer Science
Technical University of Denmark

Richard Petersens Plads
Building 324
2800 Kongens Lyngby, Denmark
www.compute.dtu.dk

Preface

This Bachelor's thesis was prepared at the Department of Applied Mathematics and Computer Science at the Technical University of Denmark in fulfillment of the requirements for acquiring a Bachelor of Science in Engineering (BSc Eng) degree in General Engineering (Cyber Systems).

Kongens Lyngby, March 27, 2026

Frederik W. L. Christoffersen

Abstract

This thesis is in the workings... [Write something when finished with all thesis chapters]

Acknowledgements

Frederik W. L. Christoffersen
General Engineering (Cyber Systems), DTU
Author of this thesis

Morten Mørup
Professor, DTU Compute
Main supervisor

Ester Salgarella
Research Fellow, AIAS Aarhus / University of Cambridge
Co-supervisor, external collaborator, and creator of SigLA

Lukas Esterle
Associate Professor, Aarhus University
Co-supervisor

Generative AI tools were occasionally used as support for brainstorming, linguistic editing, code exploration, and research during the development of this project.

Contents

Acronyms

HT Haghia Triada Tablet

SAM Segment Anything Model

mSAM MobileSAMv2

Otsu Otsu's method

GC+brush GrabCut with brush seeding

mSAM+pts MobileSAMv2 with automatic point prompts

Cefael Collections de l'École française d'Athènes en ligne

GORILA Godart and Olivier, Recueil des inscriptions en linéaire A

PDR Project Definition Report

IPR Intellectual Property Rights

GMM Gaussian Mixture Model

IoU Intersection over Union

Dice Dice-Sørensen coefficient

TP True Positives

TN True Negatives

FP False Positives

FN False Negatives

List of Figures

List of Tables

CHAPTER 1

Introduction

1.1 Background

When the British archaeologist Arthur Evans in the beginning of the 20th century went to excavate Knossos on Crete, he found a huge amount of clay tablets written in a previously unknown linear script. Some documents were clearly of a newer script, which he called *Linear B*, while others were of an older one, which he named *Linear A*. Some fifty years later, the British amateur archaeologist Michael Ventris finally succeeded in deciphering the newer script Linear B, which he proved to be representing Mycenaean Greek, the earliest attested form of Greek. But the same hypothesis did not hold for the older script Linear A, whose encoded language remained unknown, still yet to be deciphered.

A key obstacle to its decipherment has been the small size of its corpus, lately consisting of only around 2,500 inscriptions (about 1,400 if only including those well-preserved enough to be readable) - significantly smaller than that of its younger counterpart that now spans more than 4,500 inscriptions (which are comparatively longer and better preserved).[1,2] But new inscriptions are still being discovered, with the corpus most recently extended in 2025 by over 100 new documents.[3,4]

Another challenge has been the lack of digital representations of these inscriptions, in a form suitable for statistical analysis. While photographs with accompanying drawings of nearly the entire corpus are available online through Collections de l'École française d'Athènes en ligne (Cefael) (see Godart and Olivier (1976–1985)), these are only available in print

[1] Salgarella 2020.
[2] Salgarella 2025, p. 13, 46.
[3] Del Freo and Zurbach 2025.
[4] Consiglio Nazionale delle Ricerche 2025.

form. This motivated the development of the palæographical[5] database *The Signs of Linear A* (SigLA), which seeks to "[develop] a systematic, exhaustive and user-friendly open access database of all Linear A inscriptions [...] in order to carry out statistical and palæographic analyses within the epigraphic corpus" (Salgarella and Castellan 2020). However, the last addition to SigLA was in 2021, and the project has so far managed to document just 772 of all currently known inscriptions.[6,7]

1.1.1 A problem of manual digitization

The drawings currently stored on SigLA were made by Ester Salgarella using the painting tool Krita. For each document, she manually created her own annotated drawings based on the originals from Cefael. From these, she proceeded to create layered Krita files with metadata for each sign, which could then at last be imported to SigLA.[8] According to her (personal communication), this has been a highly time-consuming undertaking. And the necessity of this kind of labor-intensive work has arguably been the biggest reason to why SigLA still does not contain the entire available corpus.

The aim of this project is therefore to investigate whether parts of this process can be automated. More specifically, whether some kind of semi-automatic segmentation tool can accelerate the workflow, while maintaining high-quality outputs that are still compatible with SigLA's database.

1.1.2 A problem of small data

The small size of the Linear A corpus and the irregularities in quality of its documents is a huge problem for the application of deep learning approaches, which typically require large amounts of data to perform well. Consequently, this project will instead seek to make use of generalized

[5]The term "palæographical" refers to the study of ancient writing systems.
[6]SigLA 2021a.
[7]SigLA 2021b.
[8]Salgarella and Castellan 2020.

machine learning and computer vision, in particular Meta's state-of-the-art interactive segmentation tool, the Segment Anything Model (SAM) (Kirillov et al. 2023), which is pre-trained on a massive and diverse dataset of over 1 billion masks. SAM's generalization and interactivity (allowing user-placed markers that guide the model) might thus allow for semi-automatic annotation of Linear A inscriptions without the need for large amounts of training data, which is unfortunately not available in this context.[9]

[9]By the "interactivity" of SAM, what is meant is NOT any built-in user interface in the form of a GUI, but rather the possibility of prompt-based placement of rectangles of focus, and points of additions and deletions that guide the model's prediction.

1.2 Prior work

This thesis places itself at the intersection of computer vision, digital humanities, and epigraphy (the study of inscriptions). Since it is a thesis of engineering, computer vision will remain the primary focus, both in terms of the research questions, and the methods used to answer them. As a result, the other two fields will merely provide the context and motivation for the research. The prior work and initial literature review in focus will thus concentrate on: 1. the relevant theory and state-of-the-art within computer vision (SAM, etc.), 2. the palæographical and digital humanities work that this project will seek to support (SigLA), and 3. the overall context thereof (Linear A).

1.2.1 Research context

For the context of Linear A and its corpus, the background section above can be referred to, citing the sources for: its size (Salgarella 2020) and (Salgarella 2025), its new addition (Del Freo and Zurbach 2025), and where it's accessed (Godart and Olivier 1976–1985). And for the epigraphical and digital humanities work, the background section can also be referred to, citing the sources for SigLA: its paper (Salgarella and Castellan 2020), its about page (SigLA 2021a), and its document repository (SigLA 2021b). Here, additional context can also be provided by Ester Salgarella herself, as she is a co-supervisor of this project, and the main person behind SigLA.

1.2.2 Computer vision literature

As for the relevant theory and state-of-the-art within computer vision, the literature there can be further split into three parts; 1. sources of general image processing, 2. sources concerning SAM and its implementation, and 3. related computer vision script segmentation work for comparison.

For image processing, a general theory book (Gonzalez and Woods 2018) can be cited, as well as more method-specific papers (when other methods are used) like Otsu thresholding (Otsu 1979), and of course the

OpenCV Python documentation (OpenCV Team 2024) for short implementation reference. Then, for SAM, these are: its paper (Kirillov et al. 2023), its lighter implementation MobileSAM (C. Zhang et al. 2023) - and the newest version of that one MobileSAMv2 (Zhao et al. 2023). Lastly, for related segmentation work there exists a paper using deep learning (in contrast to the methodology of this thesis) obtaining results better than state-of-the-art (P. Zhang, Li, and Sun 2024), and a lighter EU-funded historical document segmentation tool *dhSegment* (Oliveira, Seguin, and Kaplan 2018) also using deep learning.

1.3 Problem statement

As previously stated, the aim of this project is to investigate whether parts of the process of creating annotated drawings for SigLA can be automated. More specifically, whether some kind of semi-automatic segmentation tool using interactive and generalized computer vision - specifically the Segment Anything Model (SAM) - can accelerate the workflow, while maintaining high-quality outputs that are still compatible with SigLA's database. But what would be the relative segmentation quality of such a tool? How automatic would it be exactly? And by how much could it improve the efficiency of the workflow? For a decomposition of the problem statement into its three research questions - each accompanied by their respective operationalization - see the following subsection:

1.3.1 Research questions

1. **Segmentation performance**: How well does a SAM-based segmentation approach perform quantitatively in terms of segmentation accuracy, compared to simple classical image processing baselines when applied to Linear A document images? [Operationalization: SAM-based masks will be compared to classical image-processing baselines using overlap metrics (e.g. IoU/Dice) against the ground truth segmentations derived from SigLA's Krita files for a minimum of 25 documents.]

2. **Degree of automation**: To which extent can user interaction be reduced by a SAM-based semi-automatic segmentation workflow, while still maintaining high-quality outputs? [Operationalization: The number of user interactions (e.g. clicks) required to achieve a certain level of segmentation quality (e.g. IoU/Dice) will be measured against ground truth (SigLA's Krita files) for a minimum of 25 documents.]

3. **Workflow efficiency**: How much reduction in annotation time can be achieved using the proposed tool compared to fully manual digitization, under controlled experimental conditions? [Operationalization: The time it takes Ester Salgarella to annotate manually compared to being assisted by the proposed segmentation tool will be measured for a minimum of 5 documents.]

1.4 Overview of the thesis

The rest of this thesis is structured into five main chapters (Data, Methodology, Results, Discussion, Conclusion), all split up into sections concerning each of the three research questions.

CHAPTER 2

Data

2.1 Sources

The dataset used in this thesis was constructed from photographs of raw Linear A inscriptions, and their corresponding ground truth annotations. Cefael (see Godart and Olivier 1976–1985) was chosen as the source for raw document images, since it is the only online and widely publicly available source of the printed five volume corpus - Godart and Olivier, Recueil des inscriptions en linéaire A (GORILA). As all raw document images on the pages of this corpus are accompanied by annotated drawings, an argument could be made to make use of those as ground truth annotations. However, annotations provided by Ester Salgarella from SigLA (see Salgarella and Castellan 2020) were chosen instead, as they are more up to date, and are already in a binary mask format that is suitable for comparison with segmentation model outputs.

What follows in this chapter is a detailed description of this dataset, documenting its characteristics, its construction process, and its limitations and biases.

2.1.1 Characteristics and construction

The dataset was made to consists of 32 chosen image pairs of raw documents and their corresponding ground truth annotations. Only inscriptions with clay as writing support were included, as they are the most common and representative of the Linear A corpus, and are the only ones for which SigLA ground truth annotations were already available (IS THIS TRUE?). The raw document images from Cefael are screenshots of grayscale images within the scanned printed edition of the corpus, and are thus not of the highest quality, being the only widely publicly

available images of the Linear A inscriptions. For an example of a raw document image and its corresponding ground truth annotation, see Figure 2.1.

((a)) Raw image (HT7b) **((b))** Ground truth (HT7b)

Figure 2.1: Example of a raw inscription image and its corresponding ground truth annotation.

2.1.2 Difficulty classification

The dataset was categorized into three difficulty levels (easy, medium, hard) based on the visual quality of the raw images and the clarity of the inscriptions. This classification was performed manually by inspecting each image and considering factors such as contrast, noise, and the presence of tablet breakages or general damage to the inscriptions. Concretely, the following criteria were used to classify each difficulty set (see Figure 2.2 for a summary of the criteria, and Figure 2.3 for examples of images with different difficulty levels):

- Easy: labeled as such for documents with clear ingravings, good contrast, minimal noise, and constant lighting.
- Hard: labeled to those with poor contrast, significant noise, and substantial damage making it hard even for the naked eye to read the inscriptions.
- Medium: labeled to all those inbetween the qualities of easy and hard.

Figure 2.2: Criteria for classifying the dataset into easy, medium, and hard difficulty levels.

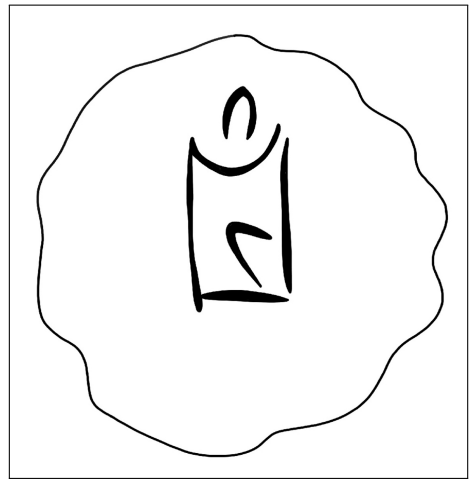
2.1.3 Image registration and preprocessing

To obtain any kind of meaningful comparison between model outputs and ground truth, the raw images and their ground truth annotations first had to be aligned and registered to each other. This was done manually in Krita by downscaling the ground truth images to the individual scales of their raw counterparts, and aligning by translation and rotation, using the raw images as background reference.

2.2 Limitations and biases

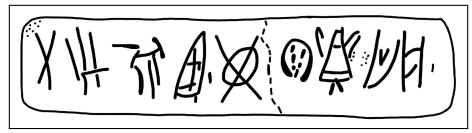
Even after image registration, ground truth images still did not perfectly align with the raw images, affecting the obtained segmentation quality. This misalignment is a source of noise in the evaluation.

((a)) Easy KHWc2104 (raw)

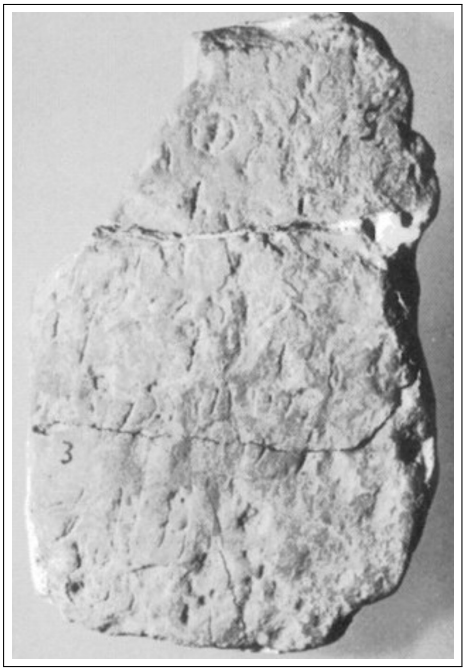


((b)) Easy KHWc2104 (gt)

((c)) Medium MA1a (raw)



((d)) Medium MA1a (gt)



((e)) Hard HT3 (raw)

((f)) Hard HT3 (ground truth)

Figure 2.3: Examples of images with different levels of segmentation difficulty: easy, medium, and hard. For each case, the raw image (left) and the corresponding ground truth annotation (right) are shown.

CHAPTER 3

Methodology

3.1 Common methodology

3.1.1 Image processing theory

[SHOULD THIS SECTION EXIST? IF SO, I SHOULD FINISH IT]

The basics of spatial filtering can be described by the following equation:

$$g(x, y) = T[f(x, y)]$$

, where $f(x, y)$ (also denoted as $I(x, y)$) is the input image, $g(x, y)$ is the output image.[1]

3.1.1.1 Binary image thresholding

$$B(x, y) = \begin{cases} 1 & \text{if } I(x, y) > T \\ 0 & \text{otherwise} \end{cases}$$

[2]

3.1.1.2 Convolutional kernels

[3]

$$(w \star I)(x, y) = \sum_{i=-a}^{a} \sum_{j=-b}^{b} w(i, j) I(x - i, y - j)$$

[1] Gonzalez and Woods 2018, p. 120.
[2] Gonzalez and Woods 2018, p. 743.
[3] Gonzalez and Woods 2018, p. 159.

where w is the convolution kernel (window) of size $m \times n$, and $a = (m-1)/2$ and $b = (n-1)/2$ are the half-width and half-height (assuming odd dimensions), respectively.[4]

3.1.2 Development environment and tools

[TODO: EXTEND THIS SECTION] Python 3.10 was used for all code development (because it has such a big library of image processing functions and machine learning model implementations), with the following libraries:

- OpenCV for image processing and computer vision tasks.
- NumPy for numerical operations and array manipulations.
- Matplotlib for data visualization.
- *Add more!*

Krita was used for manual image registration, making sure the raw image ground truth pairs were properly aligned.

3.1.3 General evaluation metrics

3.2 Segmentation quality

To answer the question of RQ1: *How does the one-shot segmentation quality of SAM compare to that of classic segmentation methods?*, the one-shot segmentation quality of a computationally efficient implementation of SAM (MobileSAMv2) was evaluated against a set of baseline models, using a suitable segmentation metric (i.e. Dice score) computed from the pairs of predicted and ground truth masks. All models were manually tuned to their best performance on the dataset, and the best performing version of SAM was compared against the best performing baseline model.

[4] Gonzalez and Woods 2018, p. 157.

The following subsections will therefore document the configuration of the baseline and state-of-the-art models, as well as the evaluation protocol and performance metrics used to answer RQ1.

3.2.1 Baseline model configuration

The baseline models against which the best SAM implementation was evaluated (from simplest to most advanced) were:

- Global thresholding (Otsu's method)
- Local adaptive thresholding (Gaussian) with bilateral pre-filtering
- Edge-based segmentation (Canny edge detection with region filling)
- Energy-based segmentation (GrabCut with brushseeds)

Together, they cover a wide range of classic segmentation techniques that might be used for binary foreground extraction. What follows is a brief description of each of these methods, including their characteristics, how they work, and most importantly their implementation details.

3.2.1.1 Global thresholding (Otsu's method)

Otsu's method is a global thresholding method that works on gray-level histograms, which was originally proposed by Otsu (1979). As the simplest of the baselines, it represents the minimum acceptable segmentation quality, since it only takes the global gray level distribution into account, with no assumptions about any spatial relationships between pixels. It is therefore expected to perform poorly on images with erosion, uneven lighting, and differences in surface texture[5], which are the exact characteristics of the images to be segmented.

Here is how it works: First, let pixels of the raw image be divided into L shades of gray $[1, 2, ..., L]$. Then, let each pixel be assigned one of two classes C_0 or C_1, guided by a threshold T, where C_0 contains the shades

[5]Gonzalez and Woods 2018, p. 751.

$[1, 2, ..., T]$, and C_1 contains the rest $[T + 1, T + 2, ..., L]$. Otsu's method then works by performing an exhaustive search over all L gray levels to find an optimal threshold T^* that results in a maximum between-class variance σ_B^2:

$$T^* = \underset{1 \leq T < L}{\arg\max}\ \sigma_B^2(T)$$

The between-class variance σ_B^2 is then defined by a function of the respective total probabilities of the two classes (ω_0 and ω_1) and the mean gray level value per class (μ_0 and μ_1):

$$\sigma_B^2(T) = \omega_0(T)\omega_1(T)[\mu_1(T) - \mu_0(T)]^2$$

For the implementation of Otsu's method in OpenCV, which takes no hyperparameters, `cv2.threshold` was used as the thresholding function, with the following two arguments:

- `cv2.THRESH_OTSU` for computation of the global Otsu threshold, and
- `cv2.THRESH_BINARY` for the production of a binary output mask, given the computed threshold.

3.2.1.2 Local adaptive thresholding (Gaussian) with bilateral pre-filtering

Local adaptive thresholding works by computing a threshold for each pixel based on its local neighborhood. This is an advantage in cases where the image has varying light conditions (which is the case for the images to be segmented), as the threshold adapts to local lighting.

Mathematically, the local threshold T is a function of a given pixel position (x, y) computed by a weighted sum of the neighborhood. The generated binary mask B is then defined as:[6]

$$B(x, y) = \begin{cases} 1 & \text{if } I(x, y) > T(x, y) \\ 0 & \text{otherwise} \end{cases}$$

[6] Gonzalez and Woods 2018, p. 761.

To compute $T(x, y)$, there are a few weighting functions to choose from. For this baseline, the Gaussian kernel was used, as it can limit the influence of distant noisy pixels by assigning weights based on distance from the kernel center. With a Gaussian kernel G_σ, the threshold $T(x, y)$ is computed as a Gaussian-weighted sum of local intensities subtracted by a constant C. Using convolution ($\star$), this can be expressed as:[7]

$$T(x, y) = (G_\sigma \star I)(x, y) - C$$

In general, the weight at an offset (i, j) from the kernel center can be written as follows (where α is a normalization constant ensuring that the kernel sums to 1, and σ controls the spatial spread of the kernel):[8]

$$G_\sigma(i, j) = \alpha \cdot \exp\left(-\frac{i^2 + j^2}{2\sigma^2}\right)$$

As a pre-filtering step, a bilateral filter (see Tomasi and Manduchi 1998) was applied to the input image before performing the Gaussian thresholding. In this way, more noise could be reduced while preserving inscriptions, as this filter is known to be effective at reducing noise while preserving edges. It takes three hyperparameters: the kernel window diameter d, the spatial standard deviation σ_{space}, and the intensity standard deviation σ_{color}. Discretely, its output intensity $I_{\text{bf}}(p)$ at a given pixel position p (where q is the position of one of its neighbors within the kernel window) can be defined as follows:[9]

$$I_{\text{bf}}(p) = \frac{\sum_q I(q)\, W_{\text{space}}(p, q)\, W_{\text{color}}(p, q)}{\sum_q W_{\text{space}}(p, q)\, W_{\text{color}}(p, q)}$$

Here, the spatial weight based on the distance between pixels p and q, and the color range weight based on the intensity difference between p and q are respectively given by:

$$W_{\text{space}}(p, q) = \exp(-\frac{\|p - q\|^2}{2\sigma^2_{\text{space}}})$$

[7] OpenCV 2024a.
[8] Gonzalez and Woods 2018, p. 167.
[9] Pan 2025.

$$W_{\text{color}}(p,q) = \exp(-\frac{\|I(p) - I(q)\|^2}{2\sigma_{\text{color}}^2})$$

For the implementation of Gaussian thresholding[10], and the bilateral pre-filter in OpenCV, the following functions were used: [11]

- `cv2.bilateralFilter` for the pre-filtering step, with hyperparameters `d` = 15, `sigmaColor` = 75, and `sigmaSpace` = 75,
- `cv2.adaptiveThreshold` as the thresholding function, with hyperparameters `blockSize` = 19, `C` = 5 (σ is determined internally based on the chosen kernel/block size), and with arguments:
  - `cv2.ADAPTIVE_THRESH_GAUSSIAN_C` for computation of the adaptive Gaussian threshold, and
  - `cv2.THRESH_BINARY` for the production of a binary output mask, given the computed threshold.

3.2.1.3 Edge-based segmentation (Canny edge detection with region filling)

Canny edge detection is a method that by itself only segments the edges of an image, excluding the enclosed regions within those edges. Here is how it works: First, the input image is smoothed using a Gaussian filter to reduce noise (using the Gaussian kernel as defined in the previous section):

$$I_s(x,y) = (G_\sigma \star I)(x,y)$$

Then, for the image gradient of each pixel, the magnitude $M_s(x,y)$ and the orientation $\theta_s(x,y)$ are computed:[12]

$$M_s(x,y) = \| \nabla I_s(x,y) \| = \sqrt{\left(\frac{\partial I_s}{\partial x}\right)^2 + \left(\frac{\partial I_s}{\partial y}\right)^2}$$

[10] OpenCV 2024b.

[11] The Gaussian kernel is *separable* and *isotropic*, meaning that it can be implemented (like done in OpenCV) as two consecutive 1-D convolutions, reducing computational complexity from $O(n^2)$ to $O(n)$ for a kernel of size $n \times n$. (Gonzalez and Woods 2018, p. 167).

[12] Gonzalez and Woods 2018, p. 730.

$$\theta_s(x, y) = \arctan\left(\frac{\partial I_s/\partial y}{\partial I_s/\partial x}\right)$$

Next, *non-maxima suppression*[13] is applied to retain only local maxima of $M_s(x, y)$ along the gradient direction $\theta_s(x, y)$, resulting in thinner edges. This is at last followed by *hysteresis thresholding*[14], classifying pixels as strong, weak, or non-edges with a low threshold T_l and a high threshold T_h, only preserving weak edges if connected to strong edges.

In an attempt to fill the regions enclosed by the detected edges, morphological closing (dilation followed by erosion) was applied to the edge mask.
For the implementation of Canny edge detection with region filling in OpenCV, the following functions were used:

- `cv2.Canny` for the edge detection method, with arguments `threshold1` $= \max(0, (1 - \sigma)v)$ and `threshold2` $= \min(255, (1 + \sigma)v)$ where v is the median pixel intensity of the input image, and σ was set to 0.1 after manual tuning.
- `cv2.morphologyEx` for the region filling method, with arguments:
  - `cv2.MORPH_CLOSE` for the closing operation, and
  - a kernel of size 15×15 for the morphological operation.

3.2.1.4 Energy-based segmentation (GrabCut with brushseeds)

GrabCut (Rother, Kolmogorov, and Blake 2004) is a classical interactive segmentation method, which makes for a good comparison to a deep learning interactive model like SAM. It is good at producing spatially coherent masks with little large-grained noise, which is ideal for the images to be segmented.

The algorithm is interactively initialized by the user, who traditionally provide a bounding box around the object of interest. Pixels outside this box are marked as definite background, while pixels inside it are

[13] Gonzalez and Woods 2018, p. 730-731.
[14] Gonzalez and Woods 2018, p. 732.

marked as initially unknown. To further guide the segmentation, the user also has the option to provide an additional number of pixels as definite foreground ($B_n = 1$), and definite background ($B_n = 0$).

As an energy-based segmentation method, optimal segmentation is achieved by minimizing a cost (energy) function E that combines a data term D and a smoothness term S, given an input image I, a binary segmentation mask B, and a set of parameters θ:[15]

$$E(B, I, \theta) = D(B, I, \theta) + S(B, I)$$

The set of parameters θ then define two *Gaussian Mixture Models (GMMs)* (probabilistic models each consisting of a weighted sum of several Gaussian distributions) that over iterations will be tuned to model the foreground and background pixel distributions. These are used to compute the data term D whose energy contribution decreases the better B matches the GMMs of foreground and background. The smoothness term S then penalizes neighboring pixels that have different labels, encouraging spatial coherence in the segmentation.

At last, the optimal segmentation B^* is found iteratively, by alternating between optimizing the GMM parameters θ to model B, and minimizing the energy function with respect to B:

$$B^* = \arg\min_B E(B, I, \theta)$$

The automatic brush seeds (see Figure 3.1 for a visualization) given to GrabCut were derived from the thresholded mask of the Gaussian adaptive thresholding baseline (see Section 3.2.1.2), as it seems to be the best at extracting foreground from the Linear A tablets, albeit with a degree of noise. The erosion of the thresholded mask was given as sure foreground, while the erosion of the inverse of the thresholded mask was given as sure background, and the rest was given as probable background. In that way, the model was provided with a good amount of guidance, while still leaving room for the model to get rid of noise and fill in gaps within the engravings of the inscriptions.
For the implementation of GrabCut in OpenCV, the following function was used:

[15] This is a simplified version of the energy function that is used by GrabCut.

- `cv2.grabCut` for the segmentation method, with arguments:
  - `iterCount` $= 5$ for the number of iterations the algorithm should run for,
  - `bgdModel` $= np.zeros((1, 65), np.float64)$ for the initial background GMM parameters,
  - `fgdModel` $= np.zeros((1, 65), np.float64)$ for the initial foreground GMM parameters,
  - `cv2.GC_INIT_WITH_MASK` for the initialization method, and
  - `mask` - a binary mask of brushseeds as the input mask, where pixels labeled as definite foreground ($B_n = 1$) were assigned the value `cv2.GC_FG`, pixels labeled as definite background ($B_n = 0$) were assigned the value `cv2.GC_BG`, and all other pixels were assigned the value `cv2.GC_PR_BG`.

3.2.2 State-of-the-art model configuration

3.2.2.1 Model architecture

Raw image → Automatic prompts → MobileSAMv2 → Binary mask

3.2.2.2 Segmentation (MobileSAMv2)

For the sake of computational efficiency, a mobile implementation of SAM - MobileSAMv2 (mSAM) (see Zhao et al. 2023) will be used instead of the full SAM model. This is a lightweight version of the original SAM model, which is designed to run efficiently on mobile devices while still maintaining good segmentation performance. It is ... but trained on the same dataset as the original SAM, and thus has the same one-shot segmentation capabilities, although with a slightly lower performance than the original.

Here is how it works: ViT encoder, prompt encoder, mask decoder, etc.

[STILL YET TO DOCUMENT HOW IT WORKS]

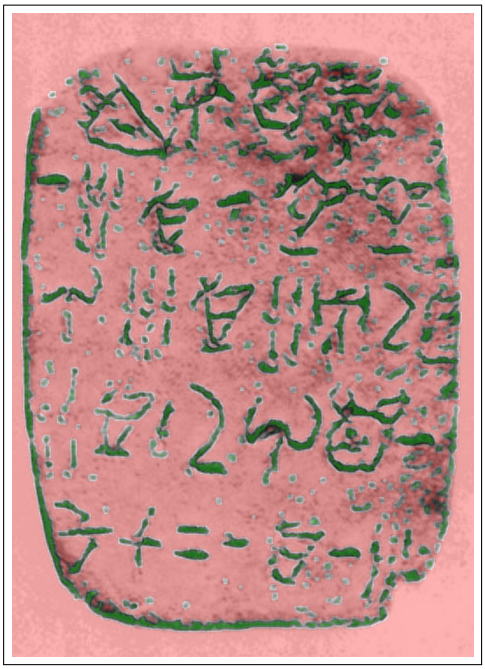


Figure 3.1: A visualization of seed distribution for GrabCut with brush seeding (GC+brush) on HT118. Green overlay represents sure foreground pixels, while red overlay represents sure background pixels, and absence of overlay represents probable background pixels.

3.2.2.3 Seed extraction

Even though the mSAM implementation already makes use of an automatic box prompt, it does so from a YOLO model trained on the same dataset as SAM. Those box prompts are generalized for and work best on natural images. Hence, they may not perform well on gray-scale images of clay tablets with Linear A inscriptions, as those were not part of the training data. Thus, mSAM needed to be adapted through the use of prompts. The model takes two types of prompts: 1. a single box prompt, and/or 2. point prompts labeled as either foreground or background. For the one-shot segmentation capability of the model, these prompts had to be automatically generated, rather than interactively placed by a user. However, as it is particularly difficult to automatically create high quality box prompts without a trained object detection model, only point

prompts were used. Therefore, two versions of MobileSAMv2 were made for comparison:

- **mSAM**: MobileSAMv2 (with the original automatic box prompts from the YOLO model),
- and **mSAM+pts**: MobileSAMv2 with automatic point prompts labeled as either foreground or background.

Just like in the case of GC+brush, the automatic point prompts for mSAM+pts (see Figure 3.2 for visualization) were derived from the thresholded mask of the Gaussian adaptive thresholding baseline (see Section 3.2.1.2), This was done by randomly sampling a number of foreground and background pixels from the mask as seeds for mSAM+pts. For the sake of simplicity, this number was kept equal for both point types to a 1000 points each, which is large enough to provide good coverage of the image, while still being computationally feasible for the model to process.

3.2.3 Evaluation protocol and performance metrics

For an evaluation of one-shot segmentation quality, relevant metrics and statistical tests need to be defined to be able to determine whether one model performed significantly better than another. More specifically, it was of particular interest to compare the best performing baseline model against the best performing version of MobileSAMv2.

The following subsections will therefore document the choice of metrics and statistical tests for the evaluation, used to answer RQ1: *How does the one-shot segmentation quality of SAM compare to that of classic segmentation methods?*.

3.2.3.1 Choice of metrics

When it comes to evaluation of segmentation models, the two most commonly used metrics are: 1. the Dice-Sørensen coefficient (Dice) and 2.

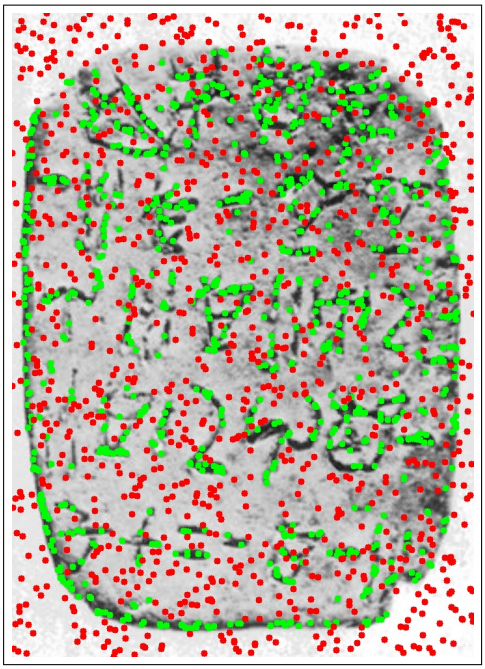


Figure 3.2: A visualization of seed distribution for MobileSAMv2 with automatic point prompts (mSAM+pts), overlayed on HT118. Green points represent foreground seeds, while red points represent background seeds.

the Intersection over Union (IoU). The Dice score is most commonly used in cases where there are heavy class imbalances (e.g. lots of background compared to regions segmented), and when foreground objects are small and thin. It is naturally more foregiving than the related segmentation metric IoU, which instead tends to be rather strict, and is specifically used to evaluate segmentation results where small displacements are critical for results. Therefore, segmentation performance was reported using Dice. Mathematically, it is defined as twice the size of the intersection of the predicted segmentation mask $\hat{Y}$ and the ground truth mask Y, divided by their summed size:[16]

[16]Mohammadi, Mollazade, and Behroozi-Khazaei 2025.

$$\text{Dice}(\hat{Y}, Y) = \frac{2 \cdot |\hat{Y} \cap Y|}{|\hat{Y}| + |Y|}$$

Thus, the raw metrics ended up consisting of the Dice score for each image, and each model.

3.2.3.2 Statistical tests

To determine whether the observed differences in scores between models were statistically significant, paired t-tests were conducted. The paired t-test has two main assumptions: 1. independence of observations, and 2. normality of residuals. The first assumption is already satisfied a priori, since the segmentation performance of one image does not directly influence the performance on another. However, the second assumption depends on results. Thus, the residuals of the paired differences in Dice scores between the two models in question needed to be checked for normality using a QQ-plot and the Shapiro-Wilk test. In case the normality assumption was violated, the non-parametric domain-specific Wilcoxon signed-rank test would be used instead.[17]

3.3 Interactive semi-automation

Here, the interactivity and quality of human-guided SAM (and GrabCut for comparison?) will be evaluated against the one-shot results from Section 3.2. [Add some context of RQ2 here, and how the question will be operationalized in bigger picture (though more detail than in introduction).]

[17] Rainio, Teuho, and Klén 2024.

3.3.1 Interactive tool architecture

3.3.2 Experimental design for automation evaluation

3.3.3 Definition of automation metrics

3.4 Workflow efficiency

[Add some context of RQ3 here, and how the question will be operationalized in bigger picture (though more detail than in introduction).]

3.4.1 Export to Krita

[It is indeed possible via the Krita Python API to export the predicted segmentation masks as Krita layers, which can then be further edited by hand by a user. The question is only if I want to do so via the API (requires Krita installed), or via a Python export script. Krita's file format is open, so both are possible.]

3.4.2 Definition of workflow efficiency metrics

3.4.3 Experimental design of the user study

3.4.4 Analysis of results

CHAPTER 4

Results and Discussion

4.1 Segmentation ability (one-shot)

After running the evaluation of the models on the dataset as illustrated in Figure 4.1, it could already be seen that no implementation of MobileSAMv2 (mSAM) was in the lead, while GC+brush seemed to outperform all. See the appendix for the full results table, and the next section for a more detailed analysis of the results.

4.1.1 Statistical significance

4.1.1.1 Normality of residuals

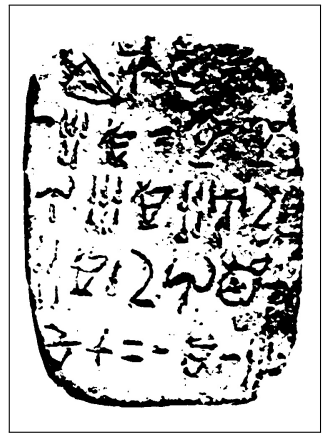
As the QQplot in Figure 4.3 shows, the residuals of the paired differences in Dice scores between GC+brush and mSAM+pts appear to be approximately normally distributed, which supports the validity of the paired t-test results. Furthermore, the Shapiro-Wilk test for normality on these residuals yields a p-value of $5.9 \cdot 10^{-1} > \alpha = 0.05$, indicating that we fail to reject the null hypothesis of normality.

4.1.1.2 Paired t-test

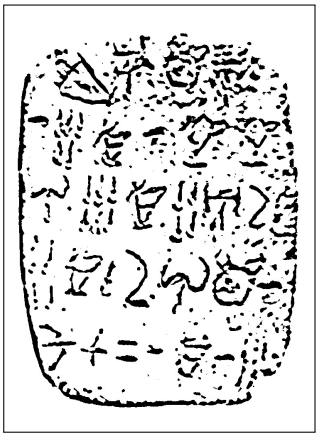
To determine whether the classic baseline model GC+brush signficiantly outperformed mSAM+pts, a paired t-test is conducted on the Dice scores obtained for each image across these two models. The null hypothesis (H_0) states that there is no difference in mean Dice scores between the two models, while the alternative hypothesis (H_1) states that GC+brush has a higher mean Dice score than mSAM+pts. Performing the paired

((a)) Raw Image

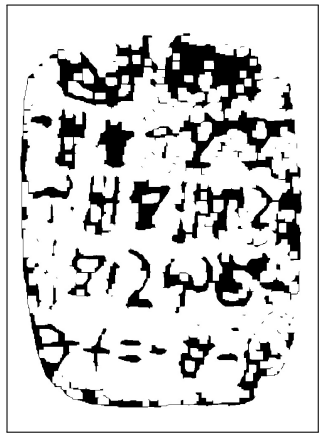
((b)) Ground Truth



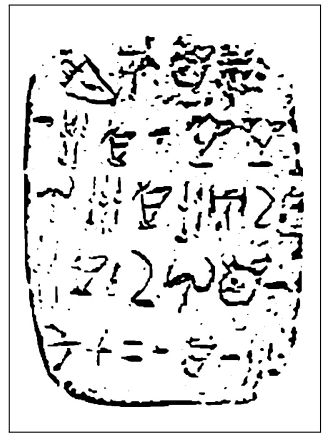
((c)) Otsu



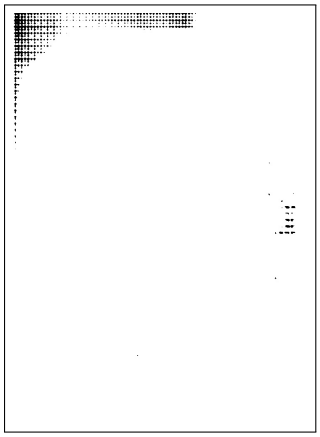
((d)) Gaussian



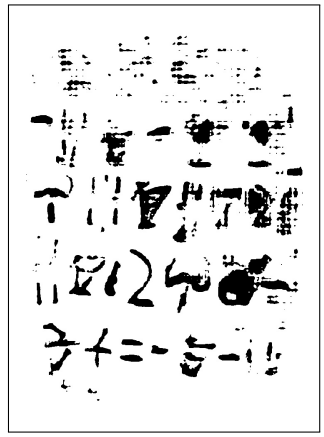
((e)) CannyFill



((f)) GC+brush



((g)) mSAM



((h)) mSAM+pts

Figure 4.1: Comparison of segmentation outputs from different models against the raw image and ground truth.

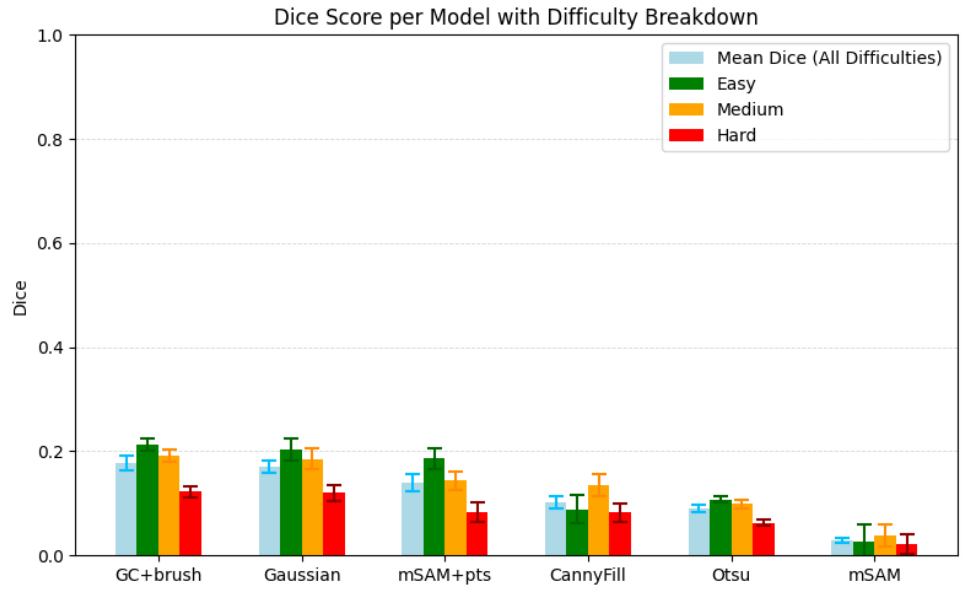


Figure 4.2: Dice Score performance per model. Split by difficulty. With error bars representing the standard error of the mean. Ordered descendingly from the left by mean performance.

t-test yielded the following results:

$$\delta = \mu_1 - \mu_2 = 0.18 - 0.14 = 0.04$$

$$t = 5.22$$

$$P(T > t) = 1.25 \cdot 10^{-5}$$

Thus, as $1.25 \cdot 10^{-5} < \alpha = 0.05$, the null hypothesis can be rejected, and it can be concluded that GC+brush significantly outperforms mSAM+pts in terms of Dice score. Hence, it can be said that mSAM underperforms in comparison to classical segmentation methods for one-shot segmentation of Linear A tablets, even when automatically prompted.

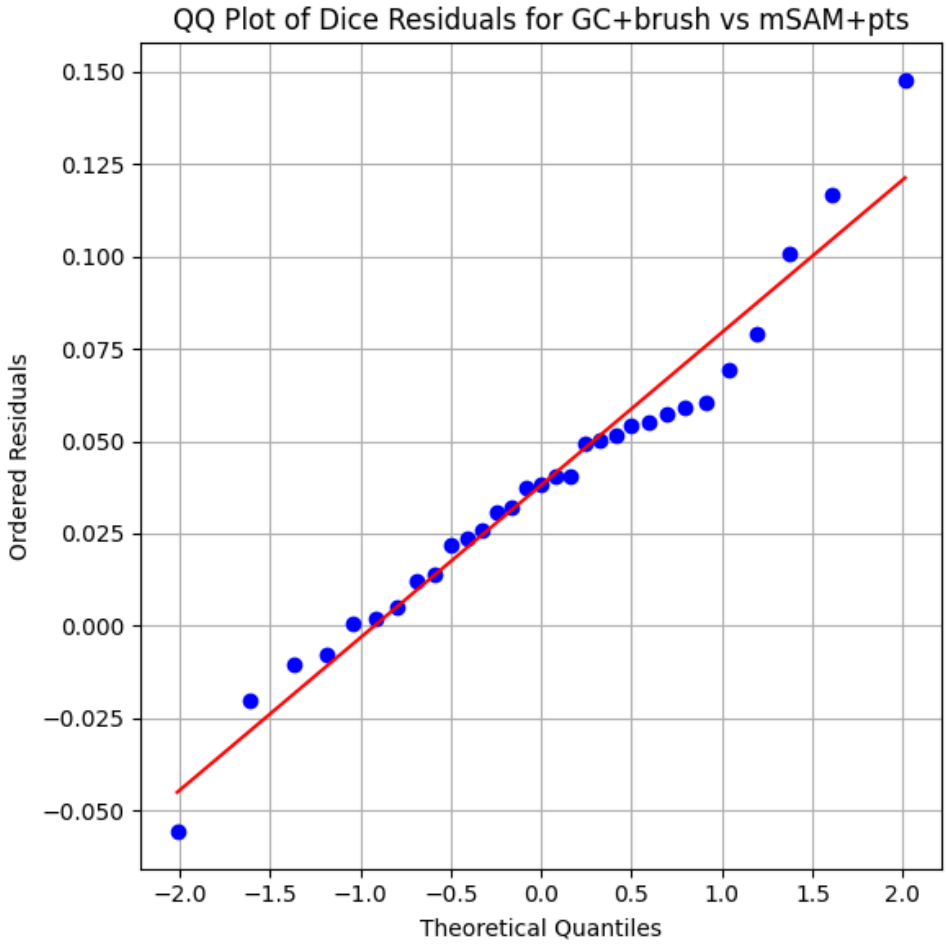


Figure 4.3: QQplot of the residuals of the paired t-test comparing GC+brush and mSAM+pts. The residuals appear to be approximately normally distributed, which supports the validity of the paired t-test results. Furthermore, the Shapiro-Wilk test for normality on these residuals yields a p-value of $5.9 \cdot 10^{-1} > \alpha = 0.05$, indicating that we fail to reject the null hypothesis of normality.

4.2 Automation and interactivity

4.3 Workflow impact

CHAPTER 5

Conclusion

Bibliography

Consiglio Nazionale delle Ricerche (2025). *A Major Addition to the Linear A Corpus*. URL: https://www.cnr.it/en/news/13529/a-major-addition-to-the-linear-a-corpus (visited on February 4, 2026).

Del Freo, Maurizio and Julien Zurbach (2025). *Recueil des inscriptions en linéaire A. Supplément 1 (RILA-S1)*. Athens: École française d'Athènes.

Godart, Louis and Jean-Pierre Olivier, editors (1976–1985). *Études crétoises. Recueil des inscriptions en linéaire A (GORILA)*. Five-volume corpus of Linear A inscriptions (GORILA I–V). Athènes. URL: https://cefael.efa.gr/result.php?site_id=1&serie_id=EtCret.

Gonzalez, Rafael C. and Richard E. Woods (2018). *Digital Image Processing*. 4th edition. New York: Pearson. ISBN: 978-0133356724. URL: https://www.cl72.org/090imagePLib/books/Gonzales,Woods-Digital.Image.Processing.4th.Edition.pdf.

Kirillov, Alexander et al. (2023). "Segment Anything." In: *arXiv preprint arXiv:2304.02643*. URL: https://arxiv.org/pdf/2304.02643.

Mohammadi, Mobin, Kaveh Mollazade, and Nasser Behroozi-Khazaei (2025). "Under- and over-segmentation: New metrics for image segmentation accuracy measurement." In: *Array* 28, page 100624. ISSN: 2590-0056. DOI: https://doi.org/10.1016/j.array.2025.100624. URL: https://www.sciencedirect.com/science/article/pii/S2590005625002516.

Oliveira, Saïd, Benoît Seguin, and Frédéric Kaplan (2018). "dhSegment: A generic deep-learning approach for document segmentation." In: *arXiv preprint arXiv:1804.10371*. URL: https://arxiv.org/abs/1804.10371.

OpenCV (2024a). *cv::AdaptiveThresholdTypes — OpenCV Documentation*. OpenCV. URL: https://docs.opencv.org/4.x/d7/d1b/

group__imgproc__misc.html#gaa42a3e6ef26247da787bf34030ed772c (visited on March 7, 2026).

OpenCV (2024b). *cv::getGaussianKernel — OpenCV Documentation*. OpenCV. URL: https://docs.opencv.org/4.x/d4/d86/group__imgproc__filter.html#gac05a120c1ae92a6060dd0db190a61afa (visited on March 7, 2026).

OpenCV Team (2024). *OpenCV-Python Image Processing Tutorials*. https://docs.opencv.org/4.x/d2/d96/tutorial_py_table_of_contents_imgproc.html. Accessed February 2026.

Otsu, Nobuyuki (1979). "A Threshold Selection Method from Gray-Level Histograms." In: *IEEE Transactions on Systems, Man, and Cybernetics* 9.1, pages 62–66. DOI: 10.1109/TSMC.1979.4310076. URL: https://engineering.purdue.edu/kak/computervision/ECE661.08/OTSU_paper.pdf.

Pan, Cong (2025). "Real-Time Hardware Architecture Design for An Innovative Bilateral Filtering Algorithm." In: *Proceedings of the 2025 5th International Conference on Automation Control, Algorithm and Intelligent Bionics (ACAIB '25)*. New York, NY, USA: ACM, pages 102–107. ISBN: 9798400714313. DOI: 10.1145/3760269.3760285. URL: https://dl.acm.org/doi/10.1145/3760269.3760285 (visited on March 23, 2026).

Rainio, Oona, Jarmo Teuho, and Riku Klén (2024). "Evaluation metrics and statistical tests for machine learning." In: *Scientific Reports* 14, page 6086. DOI: 10.1038/s41598-024-56706-x. URL: https://doi.org/10.1038/s41598-024-56706-x.

Rother, Carsten, Vladimir Kolmogorov, and Andrew Blake (2004). "GrabCut: Interactive Foreground Extraction Using Iterated Graph Cuts." In: *ACM SIGGRAPH*, pages 309–314. DOI: 10.1145/1015706.1015720.

Salgarella, Ester (2020). *Aegean Linear Script(s): Rethinking the Relationship Between Linear A and Linear B*. Cambridge: Cambridge University Press.

— (2025). *Writing in Bronze Age Crete: 'Minoan' Linear A*. Cambridge: Cambridge University Press.

Salgarella, Ester and Simon Castellan (2020). *SigLA: The Signs of Linear A: a Palaeographical Database*. Research Paper / Technical Report. Accessed: 2026-02-04. SigLA. URL: https://sigla.phis.me/paper.pdf.

SigLA (2021a). *About SigLA*. URL: https://sigla.phis.me/about.html (visited on February 4, 2026).

— (2021b). *SigLA Document Search Interface*. URL: https://sigla.phis.me/search-document.html (visited on February 4, 2026).

Tomasi, Carlo and Roberto Manduchi (1998). "Bilateral Filtering for Gray and Color Images." In: *Proceedings of the Sixth International Conference on Computer Vision*. Bombay, India: IEEE, pages 839–846. DOI: 10.1109/ICCV.1998.710815. URL: https://users.soe.ucsc.edu/~manduchi/Papers/ICCV98.pdf (visited on March 23, 2026).

Wang, Shibin et al. (2025). "OBIFlo-SAM: multi-task semantic recognition and segmentation of oracle bone inscription." In: *npj Heritage Science* 13, page 588. DOI: 10.1038/s40494-025-02157-0. URL: https://www.nature.com/articles/s40494-025-02157-0.

Zhang, Chaoning et al. (2023). "Faster Segment Anything: Towards Lightweight SAM for Mobile Applications." In: *arXiv preprint arXiv:2306.14289*. URL: https://arxiv.org/pdf/2306.14289.

Zhang, Pan, Chao Li, and Yuanhua Sun (2024). "Stone Inscription Image Segmentation Based on Stacked-UNets and GANs." In: *Discover Applied Sciences* 6.10, page 550. DOI: 10.1007/s42452-024-06264-8. URL: https://doi.org/10.1007/s42452-024-06264-8.

Zhao, Xu et al. (2023). "MobileSAMv2: Faster Segment Anything to Everything." In: *arXiv preprint arXiv:2312.09579*. URL: https://arxiv.org/pdf/2312.09579.

APPENDIX A

An Appendix

A.1 Raw segmentation performance

Table A.1: Raw Dice score data on easy images. Label column represents the name of the document segmented, while all other columns represent Dice scores obtained for those documents for a given model.

Label	CannyFill	GC+brush	Gaussian	Otsu	mSAM	mSAM+pts
HT118	0.136	0.169	0.162	0.109	0.002	0.137
HT12	0.125	0.26	0.259	0.192	0.048	0.199
HT154B	0	0.143	0.147	0.073	0.032	0.088
HT17	0.137	0.23	0.215	0.071	0.03	0.251
HT22	0.009	0.192	0.187	0.136	0.057	0.161
HT26a	0.121	0.263	0.256	0.116	0.001	0.205
HT40	0.137	0.161	0.161	0.088	0.029	0.111
HT7a	0.12	0.129	0.136	0.116	0.02	0.124
HT7b	0.175	0.18	0.162	0.074	0.026	0.123
HT90	0.017	0.179	0.181	0.14	0.012	0.154
KHWc2104	0.002	0.442	0.37	0.069	0.027	0.497

Table A.2: Raw Dice score data on medium images. Label column represents the name of the document segmented, while all other columns represent Dice scores obtained for those documents for a given model.

Label	CannyFill	GC+brush	Gaussian	Otsu	mSAM	mSAM+pts
HT10a	0.159	0.254	0.249	0.131	0.057	0.154
HT16	0.149	0.177	0.159	0.081	0.003	0.165
HT2	0.068	0.114	0.127	0.094	0.021	0.076
HT4	0.09	0.237	0.239	0.157	0.061	0.213
HT6b	0.136	0.188	0.175	0.075	0.004	0.147
HT85b	0.128	0.118	0.106	0.06	0.035	0.081
HT8b	0.228	0.274	0.267	0.11	0.052	0.157
HT94b	0.073	0.083	0.085	0.038	0.017	0.032
MA1a	0.226	0.232	0.227	0.144	0.09	0.243
PH2	0.089	0.242	0.224	0.096	0.046	0.172

Table A.3: Raw Dice score data on hard images. Label column represents the name of the document segmented, while all other columns represent Dice scores obtained for those documents for a given model.

Label	CannyFill	GC+brush	Gaussian	Otsu	mSAM	mSAM+pts
HT1	0.115	0.117	0.103	0.043	0.049	0.068
HT27a	0.137	0.153	0.148	0.059	0.047	0.005
HT3	0.074	0.08	0.089	0.058	0.046	0.078
HT30	0.104	0.096	0.098	0.054	0.006	0.082
HT33	0.039	0.054	0.051	0.016	0	0.033
HT52a	0.001	0.1	0.109	0.065	0.006	0.06
HT85a	0.125	0.084	0.097	0.057	0.006	0.092
HT8a	0.181	0.206	0.186	0.061	0.041	0.127
HT9a	0.018	0.219	0.212	0.147	0.02	0.219
HT9b	0.028	0.117	0.115	0.071	0	0.063

Table A.4: Summary Statistics per Model of Dice Score on all images.

Model	Mean Dice	Standard deviation	Standard error	n
CannyFill	0.101	0.064	0.011	31
GC+brush	0.177	0.08	0.014	31
Gaussian	0.171	0.069	0.012	31
Otsu	0.09	0.04	0.007	31
mSAM	0.029	0.023	0.004	31
mSAM+pts	0.139	0.092	0.016	31

Table A.5: Summary Statistics per Model of Dice Score on easy images.

Model	Mean Dice	Standard deviation	Standard error	n
CannyFill	0.089	0.067	0.02	11
GC+brush	0.214	0.087	0.026	11
Gaussian	0.203	0.069	0.021	11
Otsu	0.108	0.038	0.012	11
mSAM	0.026	0.017	0.005	11
mSAM+pts	0.186	0.113	0.034	11

Table A.6: Summary Statistics per Model of Dice Score on medium images.

Model	Mean Dice	Standard deviation	Standard error	n
CannyFill	0.134	0.058	0.018	10
GC+brush	0.192	0.067	0.021	10
Gaussian	0.186	0.064	0.02	10
Otsu	0.099	0.038	0.012	10
mSAM	0.039	0.028	0.009	10
mSAM+pts	0.144	0.064	0.02	10

Table A.7: Summary Statistics per Model of Dice Score on hard images.

Model	Mean Dice	Standard deviation	Standard error	n
CannyFill	0.082	0.059	0.019	10
GC+brush	0.123	0.054	0.017	10
Gaussian	0.121	0.048	0.015	10
Otsu	0.063	0.033	0.011	10
mSAM	0.022	0.021	0.007	10
mSAM+pts	0.083	0.058	0.018	10

$\sqrt{17}$
$\Omega \int \delta e^{i\pi} = -1$
{2.7182818284}ϑφερτυθιοπσδφγηξκλ
χ^2
$\sum$
λ

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Technical
University of
Denmark

Richard Petersens Plads, Building 324
2800 Kgs. Lyngby
Tlf. 4525 1700

www.compute.dtu.dk